

H3: Generated Readers and Nonlinear Predictive Coding

Architecture, sandbox evidence, and a larger-corpus experiment

CD / PC research working report

September 2026

Exploratory results; larger Colab run pending

Abstract

Can a language predictor improve by inventing useful ways to read its context, while a neural network learns how to combine their predictions? H3 explores this idea by placing a small nonlinear predictive-coding cap above a frozen transformer and a bank of statistical readers. A structural search constructs new context observers, fits next-byte distributions for them, and extends successful observers with additional distinctions. At prediction time, the cap adjusts hidden activities and reader reliability using errors on bytes that have already been observed.

On Tiny Shakespeare, H3 improves over a matched H2-style fixed-reader linear cap in all five cap initializations on both scored test segments. The clearer finding is that generated readers, including a composed reader, add value beyond the same nonlinear cap with a fixed bank. Evidence for nonlinear capacity and for predictive-coding training is more qualified: the linear generated-reader cap remains strong, and larger fitting sets strengthen but do not settle the case for nonlinearity. We give the implementation, equations, controls, numerical audits, failed objectives, costs and limitations, then specify a restartable 6.5-hour Colab experiment on enwik8. The results support a concrete architectural advance, not a general claim that predictive coding beats backpropagation or that the complete CD program has been realized.

The strongest evidence concerns useful reader generation and composition. H3's nonlinear capacity is promising with more data; its general advantage and the benefit of learned search guidance remain open.

Contents

1	What H3 is trying to accomplish	3
2	From H1 through H2 to H3	4
3	Architecture: the picture and the intuition	6
4	Generated readers as finite distinctions	7
5	The nonlinear predictive-coding cap	9
6	Sandbox protocol and audit trail	11
7	What search actually discovered	11
8	Primary H3 results	12
9	More fitting data: an exploratory follow-up	14
10	Late reader births and why refitting can matter	15
11	Historical guidance and dependency towers	16
12	Numerical verification, resource costs and limitations	16
13	Why H3 is more successful than H1: evidence and interpretation	18
14	The supplied 6.5-hour Colab experiment	18
15	Assessment and next decisions	21
A	Evidence locations and reproducibility	21

1 What H3 is trying to accomplish

A conventional transformer learns a large collection of useful computations inside a fixed parameterized architecture. The CD research program asks whether part of that computation can instead be made explicit: propose a distinction between contexts, learn what that distinction predicts, reuse it in later distinctions, and retain it if the resulting prediction improvement justifies its cost.

In this report, a *reader* is a procedure that takes the observed text prefix and returns a probability for each possible next byte. One reader may trust a transformer, another may look up a recent suffix, and another may use occurrences of the current partial word. The cap combines these distributions. It does not need to relearn from scratch that a reader’s evidence points toward the byte ‘e’ rather than ‘x’.

H3 restores two mechanisms that were simplified in H2: genuine creation of new learned readers and a nonlinear network of activities on which predictive coding can operate. It keeps H2’s useful probability interface and causal evidence-based inference. The reader graph is discovered before final cap fitting in the main experiment. A supplementary late-birth study tests growing an already trained cap, but repeated ongoing structural learning is not part of the main H3 result.

1.1 Three distinct uses of learning

1. **Statistical memory:** count which bytes follow each observed context class. This fits a reader’s distribution.
2. **Structural search:** choose which context classes to create and which successful distinctions to refine. This learns the reader graph.
3. **Neural learning and state inference:** learn how to combine readers, then adjust their reliability for a particular prefix using its recent evidence.

A good result in one of these roles does not establish superiority in the others. For example, a count dictionary can greatly improve a small transformer without showing that PC training is better than BP.

1.2 The principal evidence, before the details

The primary experiment uses 16,384 cap-fitting targets and five cap seeds. H3 scores 2.316095 and 2.392021 bits per byte on the first and following test segments. A matched H2-style linear PC cap scores 2.332998 and 2.403120. H3 wins every seed on both segments.

Against a nonlinear PC cap with the fixed reader bank, the gains are 0.012176 and 0.013580 bpb, also with five wins out of five on both segments. A reader formed by extending the first new reader contributes beyond a cap trained with the first reader alone. These comparisons are the core architectural evidence.

At this fitting-set size, however, H3 beats a linear cap supplied with the same new readers on the first segment and loses slightly on the following segment. A later exploratory study with 65,536 fitting targets improves the nonlinear comparison. It is reported separately because it was motivated by inspection of the primary test results.

2 From H1 through H2 to H3

2.1 What H1 did, and what “did not work” means

H1 already implemented a nonlinear PC-trained cap, structural growth, finite reader compositions, historical proposal guidance and a compaction demonstration. Its added readers conveyed context information through hashed categorical feature tables, followed by a neural residual writer over the byte vocabulary. Its hidden layers had widths 32 and 32; its allocated cap contained 32,064 parameters, including the feature tables and writer.

H1 was operational and its added features affected predictions. It nevertheless failed to establish a competitive advantage over a simpler fixed cap. On its original test segment, calibrated fixed BP scored 2.874994 bpb and CD-PC scored 2.877332, with fixed BP better in all three seeds. The frozen transformer scored 2.896241. Thus H1 improved the base but did not justify its additional structural/PC machinery on that comparison. Its final CD-PC graphs selected primitive copy readers rather than useful composed towers. Historical guidance was essentially neutral.

H1’s deployment computation also offered little distinct evidence for PC to reconcile: its free prediction was essentially a forward computation. The PC procedure supplied a form of training credit, rather than adding substantial new observations during inference.

2.2 What H2 established

H2 changed the evidence interface. Thirteen readers emitted full next-byte distributions: the transformer, eight smoothed suffix readers, three copy readers, and one copy reader with a suffix fallback. A small gate learned their reliability and adjusted it using their performance on the last 16 observed bytes.

This was a much stronger predictor. The original H2 all-reader PC result was 2.332323 bpb on the first segment; its five-reader compression-first version scored 2.335741. Compression-first selection beat an update-matched fixed-five control, but the full bank remained slightly better. H2 therefore demonstrated useful restricted structural selection, with a small predictive cost for fewer active readouts.

The large change from H1 is not an isolated interface ablation. H2 also introduced substantial count memory fitted on 700,000 training bytes. It is reasonable to infer that preserving predictive evidence helped, but the entire H1-to-H2 improvement cannot be assigned to that one change.

2.3 What is new in H3

Aspect	H1	H2	H3
Reader interface	Hashed observer features feeding a neural correction	Full byte distributions from a fixed bank	Full distributions from fixed and newly constructed readers
Structural operation	Add finite observers; compositions available	Select or prune among 13 fixed readers	Fit new reader tables; append distinctions to accepted programs
Neural computation	Nonlinear residual cap	Linear reliability prior	Two nonlinear hidden activity layers plus reliability state
Free inference	Essentially feedforward at deployment	Settle reliability using observed-prefix evidence	Settle both hidden layers and reliability using that evidence
Learning comparison	PC versus BP	PC versus BP with shared free inference	Centered local PC versus BP through the same finite inference
Strongest result	Components run; fixed cap remains better	Strong memory/interface; useful restricted selection	Generated and composed readers consistently add value
Guide evidence	Small-history benefit unclear	Mainly graph reuse	Prospective audit only; no demonstrated ranking gain

Table 1: The conceptual progression. “H2-style” controls in H3 are refitted under H3’s common schedule; they are not copied historical H2 scores.

H3’s main fixed-bank nonlinear cap has 18,285 parameters. Adding its two accepted readers increases that to 19,183. Although the nonlinear cap is much larger than H2’s 1,521-parameter gate, the added-reader comparison changes its dense parameter count by only 898. New count tables and vocabulary are additional memory and must also be charged.

3 Architecture: the picture and the intuition

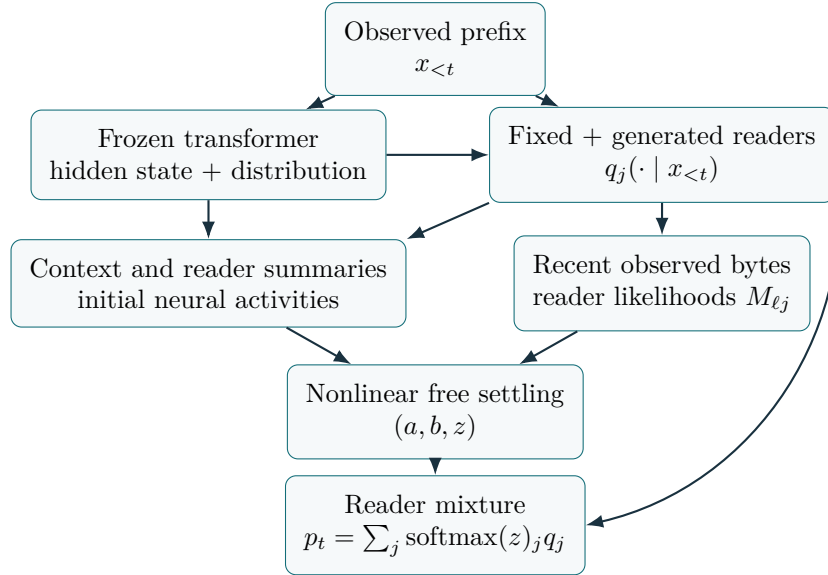


Figure 1: Prediction-time computation. The unknown next byte is absent from the free settling phase. Reader-table fitting and structural search happen on training data before this diagram is deployed.

Think of the cap as a small committee chair. Each reader hands it an explicit probability distribution. The chair first uses the current context to form an opinion about whose advice to trust. It then checks how the readers explained the recent observed text and adjusts its internal state. The adjusted reliability weights combine the next-byte predictions.

For an illustrative prefix such as **the c**, a partial-word reader groups it with other contexts whose current partial word is **c**. A refined reader additionally observes that the preceding word is **the**. Its next-byte counts may differ from contexts such as **I c**. The neural cap can learn when this lexical evidence is useful relative to suffix or copy evidence. This example explains the mechanism; it is not a claim about measured probabilities for that phrase.

3.1 How this fits the generalized-transformer view

At an abstract level, all three versions instantiate a repeated process of reading context, constructing proposals, reconciling evidence, and writing a predictive state. In H3, a reader supplies a point in the probability simplex

$$\Delta^{255} = \{p \in \mathbb{R}^{256} : p_y \geq 0, \sum_y p_y = 1\}.$$

The cap writes a point in the convex hull of those reader distributions. Its Euclidean activity coordinates (a, b, z) move under a scalar energy, while softmax maps reliability coordinates to mixture weights. Structural growth adds a new available point-valued function to this geometry.

This is a concrete special case of the broader generalized-transformer proposal, not a claim that transformer attention and this energy are mathematically identical. A standard transformer writes learned feature vectors through attention and nonlinear blocks; H3's final output is constrained to a mixture of reader distributions.

4 Generated readers as finite distinctions

4.1 Observers and refinement

Let $c = x_{<t}$ denote an observed context. An observer program ϕ assigns a discrete key $\phi(c)$. It induces an equivalence relation

$$c \sim_{\phi} c' \iff \phi(c) = \phi(c').$$

Adding another observer ψ forms $\phi' = (\phi, \psi)$, which refines this partition: contexts formerly grouped together can now be distinguished. This gives an executable finite interpretation of a distinction and its composition.

The implemented grammar has 16 atoms: literal bytes at lags 1–12; the current partial word; the preceding whitespace-delimited word; current partial-word length capped at 12; and the class of the last byte (whitespace, uppercase, lowercase, digit or other). Words retain at most their last 12 bytes and are observed within a 64-byte causal window. The word vocabulary is fitted and frozen on the count-training prefix. Unknown words map to a reserved ID.

Programs select one to three atoms. Each atom ID fits in 21 bits and the tuple is packed into an unsigned 64-bit key, avoiding hash collisions within these bounds. The initial pool contains the 16 atoms plus three lag combinations. After a program is accepted, candidates can append another atom to it. A parent pointer supplies a probabilistic backoff graph as well as a reusable program construction path.

Scope. Word segmentation and the atomic grammar are supplied by the implementation; search does not discover them from nothing. This is a bounded observer/refinement system, not a full STLM or an exact distinction-calculus quotient engine. A word-prefix child augmented with previous word formally refines its parent partition. The suffix-6 fallback of the first reader is a probabilistic fallback, not evidence that word-prefix observation refines the suffix-6 partition.

4.2 Learning the distribution of a new reader

For reader r , let $C_r(k, y)$ count target byte y following key k , with $n_r(k) = \sum_y C_r(k, y)$. Counts are fitted only on the initial 700,000 training bytes in the sandbox. Keys require support at least four; at most 12,000 keys are retained per reader, ordered by support.

With $\epsilon = 1/256$ and $\alpha = 16$, the reader predicts

$$q_r(y \mid c) = \frac{\alpha q_{\text{parent}}(y \mid c) + (1 - \epsilon)C_r(\phi_r(c), y) + \epsilon n_r(\phi_r(c))/256}{\alpha + n_r(\phi_r(c))}. \quad (1)$$

The first generated reader backs off to the inherited suffix-6 reader. A refined reader backs off to its accepted parent. If a key has no retained observations, $n_r = 0$ and prediction equals the parent exactly.

Proposition 1 (Normalized and bounded reader outputs). *If the parent is a probability distribution with every coordinate at least $\epsilon/256$, then the child defined by Equation 1 is also such a distribution.*

Proof. The numerator sums to $\alpha + n_r$. Its uniform contribution plus the parent’s lower bound is at least $(\alpha + n_r)\epsilon/256$ in each coordinate. Dividing proves both statements. For an absent key, the count contribution vanishes. $\square$

The probability floor makes log losses finite: with the inherited smoothing, each single-byte loss is at most 16 bits. It does not supply an adaptive-search generalization guarantee.

4.3 The retained search objective, and a failed alternative

An initial global EM mixture of the 13 old readers defines p . For a new candidate q_r , screening considers

$$p_{r,\gamma} = (1 - \gamma)p + \gamma q_r, \quad \gamma \in \{0.03, 0.1, 0.2, 0.35, 0.5\}.$$

The insertion weight is selected on the fitting targets. Its separate outcome score is

$$\hat{g}(r) = \frac{1}{|O|} \sum_{t \in O} \log_2 \frac{p_{r,\gamma}(y_t | c_t)}{p(y_t | c_t)}. \quad (2)$$

The sandbox accepts the best candidate when $\hat{g} > 0.0002$ bpb, subject to a 16-million-bit estimated-description ceiling per reader and at most four births. The retained probability mixture becomes the next search state.

The first attempted score was $\hat{g}(r) - \hat{C}(r)/(50\,000\,000)$. It rejected every candidate. The code-size proxy was

$$\hat{C}(r) = 64 + 21d + 21dK + 24Z,$$

where d is tuple length, K the number of stored keys and Z the number of nonzero next-byte counts. It excludes vocabulary cost and is not an actual complete prefix code or a serialized-memory measurement.

The revised objective separates prediction improvement from a storage constraint. This avoids insisting that every useful module repay itself over a short arbitrary horizon. It also changes the claim: H3 demonstrates improved prediction under a capacity ceiling, *not* net compression of the 1.1 MB corpus after all model bits are charged.

If a reader of estimated size C maintained a per-target gain $g > 0$, its idealized repayment horizon would be $N = C/g$. This is an accounting identity under fixed gains, not a theorem that the measured gain persists. The two accepted readers imply roughly 282 million and 895 million future targets by this proxy. Actual storage, deployment frequency, distribution shift and interactions alter that calculation.

Algorithm 1 Staged H3 reader construction (sandbox implementation)

Require: Fixed readers Q , count prefix D , disjoint fit/outcome sets F, O

- 1: Fit an EM mixture p of Q on F ; set accepted graph $G \leftarrow \emptyset$
 - 2: **for** at most four birth rounds **do**
 - 3: Propose initial programs and one-atom refinements of programs in G
 - 4: **for** each proposed program r **do**
 - 5: Fit bounded counts on D ; construct its parent-backed distribution
 - 6: Select insertion weight γ_r on F
 - 7: Measure outcome gain on O ; record size, support, parent and gain
 - 8: **end for**
 - 9: Choose the greatest-gain candidate satisfying the size ceiling
 - 10: **if** no eligible candidate has gain > 0.0002 **then**
 - 11: **break**
 - 12: **end if**
 - 13: Add the candidate to G ; update $p \leftarrow (1 - \gamma_r)p + \gamma_r q_r$
 - 14: **end for**
 - 15: Fit final neural caps using fixed readers Q and selected graph G
-

This outcome set is reused adaptively. It is separate from validation and test, but repeated screening still creates selection bias. Final paired test comparisons assess the selected system; the screening gains themselves are not unbiased independent confirmations.

5 The nonlinear predictive-coding cap

5.1 Inputs and the free energy

Let J be the number of readers. The cap input contains the frozen transformer hidden state and four summaries per reader: normalized entropy, peak probability, log support and agreement with the transformer’s most likely byte. The feature vector is standardized using cap-fitting data, with standard-deviation floor 0.05 and clipping to $[-8, 8]$. The current target is not an input.

For the sandbox, the hidden activity widths are 96 and 64. Given normalized features x , define the layer predictions

$$f_1(x) = \tanh(W_1x + b_1), \quad f_2(a) = \tanh(W_2a + b_2), \quad f_3(b) = W_3b + b_3.$$

The activities a, b and reliability logits $z \in \mathbb{R}^J$ initialize from this forward pass. Let $M_{\ell j}$ be reader j ’s probability of the ℓ th recent *observed* byte, evaluated from its earlier prefix. With $L = 16$ and $w(z) = \text{softmax}(z)$, free energy is

$$E_0(a, b, z; x, M) = \frac{1}{2}\|a - f_1(x)\|^2 + \frac{1}{2}\|b - f_2(a)\|^2 + \frac{1}{2}\|z - f_3(b)\|^2 - \frac{\lambda}{L} \sum_{\ell=1}^L \log \left(\sum_{j=1}^J w_j(z) M_{\ell j} \right), \quad \lambda = 2. \quad (3)$$

The first three terms discourage the activities from departing too far from the learned network. The last term rewards explanations of the recent observed text. Thus inference can bend the network’s initial prediction toward readers that fit the local evidence.

The implementation takes 32 simultaneous gradient-descent steps with step size 0.15. All three activity levels move. The final predictive distribution is

$$p(y | c) = \sum_{j=1}^J w_j(z^{\text{free}}) q_j(y | c). \quad (4)$$

Causality. At the moment of predicting y_t , the features, current reader distributions and M are functions of $x_{<t}$. Probabilities of already observed bytes are legal evidence; y_t is used only to score a prediction or to construct the training nudge. The tests alter the target and future while holding the prefix fixed and verify that prediction inputs and free inference do not change.

5.2 Explicit activity updates

Let $e_1 = a - f_1(x)$, $e_2 = b - f_2(a)$ and $e_3 = z - f_3(b)$. For any reader-likelihood vector v , define the responsibilities

$$\rho_j(z; v) = \frac{w_j(z) v_j}{\sum_k w_k(z) v_k}.$$

Then the memory-term gradient is

$$g_M(z) = \lambda \left(w(z) - \frac{1}{L} \sum_{\ell} \rho(z; M_{\ell}) \right).$$

The simultaneous free-phase gradients are

$$\begin{aligned} \nabla_a E_0 &= e_1 - W_2^{\top} [e_2 \odot (1 - f_2(a)^2)], \\ \nabla_b E_0 &= e_2 - W_3^{\top} e_3, \\ \nabla_z E_0 &= e_3 + g_M(z). \end{aligned}$$

These equations explain what “nonlinear PC” means in this implementation: feedback from reliability errors changes b , then errors in b change a , while local prediction errors pull each activity toward its layer prediction.

5.3 Training: centered PC and the BP control

For an observed training target y , the nudge loss uses natural logarithms:

$$C(z; y) = -\log \left(\sum_j w_j(z) q_j(y | c) \right), \quad E_\beta = E_0 + \beta C.$$

After free settling, positive and negative phases each run 32 additional steps from the free state, with $\beta = 0.05$ and -0.05 . The parameter gradient estimate is

$$\widehat{\nabla_\theta C} = \frac{\partial_\theta E_{+\beta}(s^+) - \partial_\theta E_{-\beta}(s^-)}{2\beta}. \quad (5)$$

Here partial derivatives hold activities fixed. For example,

$$\partial_{W_2} E_0 = -[e_2 \odot (1 - f_2(a)^2)] a^\top, \quad \partial_{W_3} E_0 = -e_3 b^\top.$$

The implementation writes these local partials directly into optimizer gradients; it does not backpropagate through the settling trajectory for PC training. It uses AdamW, weight decay 0.01, batch size 256 and gradient clipping at norm one. The selected sandbox learning rate is 0.003.

The BP control uses the same reader graph, feature interface, widths, free energy and 32-step free inference. It differentiates the final mixture loss through those 32 steps. This is a strong control: it retains inference and changes the learning rule.

The familiar equilibrium-propagation connection requires smooth, stable equilibrium branches and sufficiently converged free and nudged states [4]. Under those assumptions and $\beta \rightarrow 0$, the energy-partial contrast gives the equilibrium objective gradient. Centering reduces finite-nudge bias for smooth branches. It does not remove finite-settling error. At 32 steps the PC update approximates an equilibrium gradient, whereas BP differentiates the exact finite computation; these are not identically the same gradient.

Algorithm 2 One minibatch update and a target-free prediction

- 1: Construct prefix-only features x , evidence M and current reader distributions
 - 2: Initialize $s = (a, b, z)$ by the neural forward pass
 - 3: $s^0 \leftarrow$ 32 descent steps on E_0
 - 4: **if** prediction only **then**
 - 5: **return** the mixture in Equation 4
 - 6: **end if**
 - 7: **if** PC training **then**
 - 8: $s^+ \leftarrow$ 32 steps on $E_{+\beta}$ starting at s^0
 - 9: $s^- \leftarrow$ 32 steps on $E_{-\beta}$ starting at s^0
 - 10: Compute the local centered parameter contrast in Equation 5
 - 11: **else**
 - 12: Differentiate the target loss through the finite free-settling computation
 - 13: **end if**
 - 14: Clip the gradient; apply AdamW; checkpoint parameters and optimizer state
-

6 Sandbox protocol and audit trail

6.1 Data, base and selection

All sandbox studies use the same 1,115,394-byte Tiny Shakespeare file with a contiguous 90/5/5 split. H3 inherits a frozen byte transformer with two blocks, width 64, four heads and context length 64. It has **120,576 distinct parameters**; an earlier H1 count of 136,960 double-counted a tied matrix and is corrected here.

Reader counts use the first 700,000 training bytes. Cap fitting uses 16,384 later training targets; the outcome set contains 4,096 disjoint later targets. The first validation half supplies 4,096 development targets and the second supplies 4,096 checkpoint-selection targets. Final scoring uses the first 32,768 targets after the test context offset, followed by the remaining 22,938 targets. The transformer had already used the broader training region and validation in earlier work.

Six development fits compared learning rates 0.001 and 0.003 for fixed nonlinear PC, generated nonlinear PC and generated BP at seed 101. Their mean development scores were 2.377239 and 2.373021 respectively, selecting 0.003. The final study used seeds 7, 19, 37, 53 and 71, eight arms per seed, 480 optimizer updates per fit, and selection checks every 40 updates. Checkpoint hashes were locked before scoring the primary final models.

6.2 What the controls isolate

- **H2-style linear PC:** refitted all-13-reader baseline under the H3 schedule and centered-PC implementation, not the historical H2 672-update number.
- **Fixed nonlinear PC:** same nonlinear architecture without generated readers.
- **Generated linear PC:** supplies the new readers to a strong low-capacity cap.
- **Generated BP:** same nonlinear structure and free inference, different training credit.
- **First-reader-only PC:** retrains with the first accepted program but omits its child.
- **Retrained no-inference cap:** fits without prefix-evidence settling; stronger than a frozen switch-off diagnostic.
- **Random additions:** same number of additional readouts and the same table-fitting rules, but different programs. This is not perfectly table-memory matched, and it can reference an accepted parent; it is not an independent random-search trajectory.

The five seeds vary cap fitting, not the base, corpus, dictionary or discovered graph. Both the corpus and original test region have been used repeatedly during exploratory research. The following segment adds within-corpus evidence, not an independent domain.

7 What search actually discovered

Search evaluated 98 candidate programs over three rounds. Two were accepted:

Program	Keys	Nonzero counts	Proxy bits	Outcome gain
Current word prefix	11,306	29,661	949,375	0.003364
Word prefix + previous word	12,000	53,626	1,791,130	0.002001

Table 2: Newly fitted readers; gains are scalar insertion-screen gains in bpb, not their final neural-cap contributions. Both accepted insertion weights were 0.1.

The first reader backs off to suffix-6. The second backs off to the first and is constructed by adjoining the preceding-word distinction. Retained key coverage on the fitting examples was about 87.2% and 34.2% respectively. The third round’s best outcome gain was 0.000143, below the acceptance threshold, so growth stopped.

The successful child is a genuine composition in this bounded grammar. Its key combines two observations, its statistical table is newly learned, and its fallback reuses the parent. This goes beyond H2’s subset selection among already available readers. Its benefit must nevertheless be measured by retraining a cap with and without it; the two insertion gains cannot simply be added to predict the final cap gain.

Search time was approximately 12.6 seconds on the sandbox CPU, excluding construction of the observer matrix, vocabulary and common caches. This is not an end-to-end deployment-time estimate.

8 Primary H3 results

8.1 Absolute predictive scores

Condition	First test bpb	Following bpb
H2-style linear PC	2.332998	2.403120
Fixed-reader nonlinear PC	2.328272	2.405601
H3: generated nonlinear PC	2.316095	2.392021
Generated-reader BP	2.319070	2.394462
Generated-reader linear PC	2.317770	2.390523
First generated reader only	2.323444	2.397026
Retrained without settling	2.317710	2.392786
Random reader additions	2.326272	2.399372

Table 3: Mean of five cap seeds, 16,384 fitting targets and 480 updates. Lower is better. All rows use the same scored targets and selection protocol.

H3 gains 0.016903 bpb over the matched H2-style control on the first segment, about 0.72% of that control’s cross entropy. This is a modest incremental improvement on top of a strong memory-assisted baseline, rather than a repetition of the much larger H1-to-H2 jump.

8.2 Paired contributions

Comparator	First gain	Wins	Following gain	Wins
H2-style linear PC	+0.016903	5/5	+0.011099	5/5
Fixed-reader nonlinear PC	+0.012176	5/5	+0.013580	5/5
Generated-reader BP	+0.002974	5/5	+0.002441	4/5
Generated-reader linear PC	+0.001675	4/5	-0.001498	1/5
Retrained without settling	+0.001614	4/5	+0.000765	2/5
Random reader additions	+0.010177	5/5	+0.007351	5/5
First generated reader only	+0.007349	5/5	+0.005005	5/5

Table 4: Comparator loss minus H3 loss; positive favors H3. Wins count cap seeds.

The fixed-nonlinear and first-reader-only comparisons are particularly informative. New readers improve the same nonlinear architecture in every seed on both segments; the child improves beyond its parent alone in every seed on both segments. Random additions do worse. These results support useful program selection and reuse within the implemented grammar.

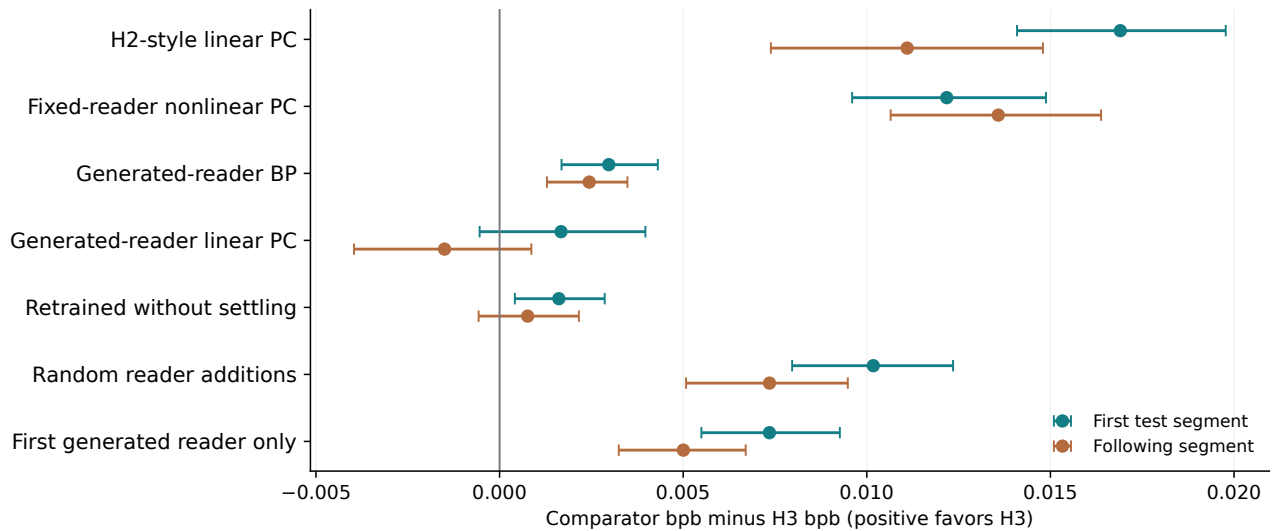


Figure 2: Primary paired gains with descriptive 95% block-bootstrap intervals. Blocks contain 256 contiguous targets; differences are first averaged across seeds. The intervals describe these scored text regions, not independent-corpus or search-selection uncertainty.

The capacity comparison is weaker. Generated nonlinear PC gains 0.001675 bpb over generated linear PC on the first segment but loses 0.001498 on the following segment; both corresponding intervals cross zero. Thus the primary study does not justify saying that a bigger nonlinear cap is consistently better. The PC–BP mean difference favors PC, with five first-segment wins and four following-segment wins. This is evidence that the implemented local update is competitive here, not that PC is intrinsically a better credit mechanism across data, budgets or architectures.

8.3 Prediction-time settling

For the trained H3 model, switching settling off increases loss by 0.003085 bpb on the first segment and 0.001823 on the following segment. Replacing the correct prefix evidence with another context’s evidence increases loss by 0.002308 and 0.000739 respectively. These frozen-model interventions show that the deployed state inference uses local evidence.

The separately retrained no-inference model is closer: H3 gains 0.001614 and 0.000765 bpb, with four of five and two of five seed wins. The second interval crosses zero. This distinction matters: an intervention can show dependence on a component without proving that retraining a simpler system cannot recover most of the benefit.

9 More fitting data: an exploratory follow-up

The mixed capacity result motivated a second study using 65,536 fitting targets, four times as many distinct targets, while excluding all original outcome targets. It retained the same discovered graph, count-training prefix, learning rate and 480-update budget. Five conditions used three seeds: 7, 19 and 37.

This study was initiated after examining primary test results. Its scores reuse the same test regions and are exploratory evidence, not fresh confirmation.

Condition	First test bpb	Following bpb
H2-style linear PC	2.322779	2.398880
Fixed-reader nonlinear PC	2.318312	2.399844
Generated-reader linear PC	2.311881	2.385855
H3: generated nonlinear PC	2.305708	2.384044
Generated-reader BP	2.306608	2.385739

Table 5: Larger fitting set, three seeds, unchanged reader graph and optimizer-update count.

Comparator	First gain	Wins	Following gain	Wins
H2-style linear PC	+0.017071	3/3	+0.014835	3/3
Fixed-reader nonlinear PC	+0.012604	3/3	+0.015799	3/3
Generated-reader BP	+0.000900	2/3	+0.001695	2/3
Generated-reader linear PC	+0.006172	3/3	+0.001810	2/3

Table 6: Paired gains over the larger-fitting-set controls.

H3’s advantage over the generated linear cap grows to 0.006172 bpb on the first segment (three wins out of three) and 0.001810 on the following segment (two wins out of three). The first block interval is $[0.004233, 0.008129]$; the following interval is $[-0.000405, 0.003941]$.

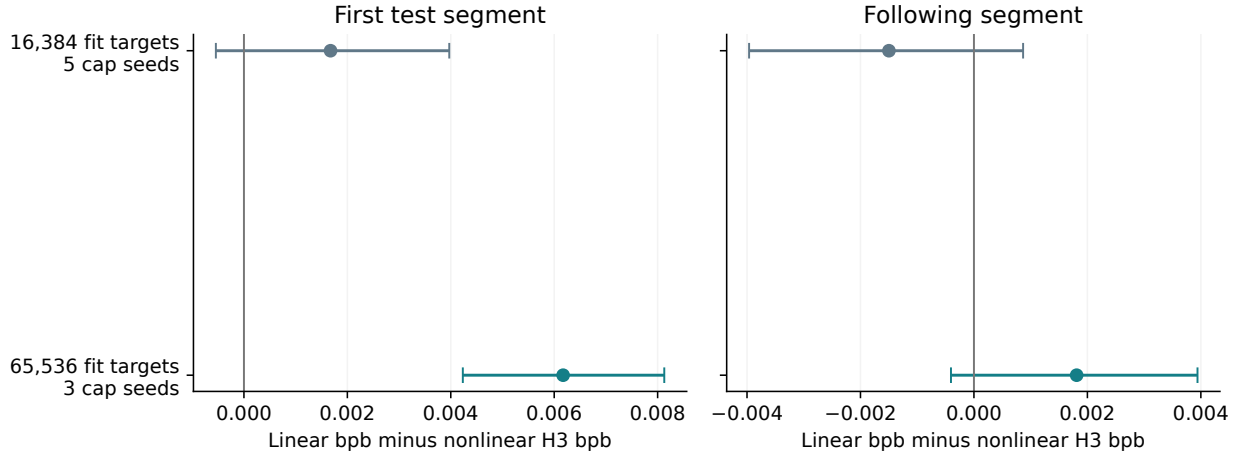


Figure 3: The nonlinear-cap comparison strengthens with more unique fitting targets. The studies differ in seed count and were not planned as an untouched confirmatory data-scaling curve.

The change concerns data diversity, not more optimizer updates. With 480 batches of 256, both recipes draw 122,880 training examples with replacement. Increasing the available pool reduces repeated exposure to the same examples and can help estimate a richer gating function. This is a plausible explanation of the improved capacity comparison, not a demonstrated scaling law or a sample-complexity threshold.

The mean PC advantage over BP is now 0.000900 and 0.001695 bpb, with two wins out of three on each segment. The first-segment interval crosses zero. Larger data makes the capacity question more interesting, but does not establish general PC superiority.

10 Late reader births and why refitting can matter

The staged scalar screen is cheap, but it can reject a reader whose value depends on learning when to trust it. A development-only experiment exposed this limitation. Starting from an already fitted fixed-reader nonlinear PC cap, every one of the 19 root proposals failed scalar insertion; the best outcome gain was -0.000038 bpb and the word-prefix proposal scored -0.000793 .

The cap was then actually enlarged and refitted for 160 updates at learning rate 0.001, alongside an equally continued no-birth control. All branches restarted AdamW. New input/output connections started at zero and the new reader bias at -4 , making the initial function approximately, rather than exactly, preserved.

Continued model	Best first-validation-half bpb
No birth	2.380823
Add first reader	2.380056
Add reader tower	2.379146

Table 7: One developmental initialization. These are validation-selected scores, not additional independent final-test evidence.

The result illustrates a mechanism: adding a reader creates a new conditional routing opportunity. A

single global insertion weight cannot express that opportunity, whereas a refitted cap can. It motivates a future loop of proposal, short contextual refit, comparison with a no-growth continuation, and commit. It does not establish that such a repeated loop already works at scale.

This also helps clarify the objective change. Two distinct barriers can reject useful structure: charging an unrealistic short repayment horizon and evaluating only a frozen scalar insertion. Relaxing the first helped the main H3 search; the late-birth test shows why the second remains a limitation.

11 Historical guidance and dependency towers

H3 supports dependency towers in a narrow, operational sense: accepted reader programs can serve as parents of new programs, and their predicted distributions supply backoff values. The accepted word-prefix/previous-word reader demonstrates one such extension.

The historical search guide is much less established. The H3 audit fits a ridge regressor using previous-round labels and four candidate descriptors: fitting gain, support fraction, tuple length and estimated code cost. It predicts outcome gain for the next round. With 19 earlier labels, the best gain in four guide-nominated candidates tied the residual-only ranking at 0.002001. With 52 earlier labels, the guide’s best was 0.000087, below the residual-only 0.000143.

This guide did not choose the deployed graph. There were only three closely related search states, too little diversity to support a strong learning-to-search claim. H1 collected more trial labels, but still too few independent states and trajectories to demonstrate a reliable guide advantage.

A meaningful next guide study should label *contextually refitted* candidate gains relative to equally trained no-growth controls; vary data regions, cap states and perhaps corpora; then evaluate candidate ranking on held-out search trajectories. Candidate count alone is not the effective sample size when many labels share nearly the same state. The supplied larger notebook retains an audit-only guide and saves all labels; it does not rely on unproven guide ranking to cut search.

12 Numerical verification, resource costs and limitations

12.1 What was checked in the H3 sandbox

Six original H3 tests verify local energy partials, activity gradients, energy descent and target-free inference, weak-nudge agreement with BP, scalar/vector reader agreement and normalization, and exact optimizer continuation. A separately implemented target-free full-distribution predictor agrees with cached probabilities to maximum absolute difference 1.19×10^{-7} on checked development examples.

A small-nudge double-precision equilibrium test gives relative gradient error about 7.4×10^{-8} . This limit check is distinct from behavior at the deployed 32-step budget. The trained-state diagnostic is:

Settling steps	PC/BP cosine	Relative error	Stationarity RMS
32	0.941012	0.356001	0.053657
64	0.972212	0.249575	0.033756
128	0.992280	0.136237	0.015418

Table 8: A trained development batch. More settling improves agreement; default inference is not fully converged.

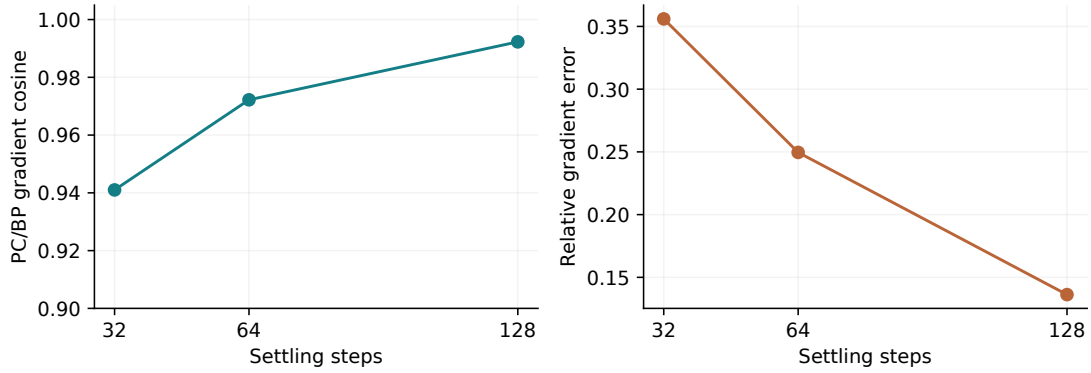


Figure 4: Finite settling is a substantive approximation, not just a theoretical footnote.

All 55 final model checkpoints from the primary and larger-fit studies were verified. The release audit recomputed 270 mean losses from retained finite per-target arrays. This guards against transcription and aggregation errors; it does not remove experimental selection effects.

12.2 Memory and time accounting

The inherited dictionary stores 89,746 contexts and 388,416 nonzero next-byte entries, serialized in 23,042,029 bytes (about 22 MiB). The two generated readers add 23,306 keys and 83,287 nonzero counts, plus a shared vocabulary of 42,958 entries. The added reader/vocabulary deployment pickle is 1,574,180 bytes; added count arrays are 1,039,200 bytes. The 2,740,505-bit description proxy is much smaller because it excludes the vocabulary and runtime representation. These quantities are not interchangeable.

Mean primary CPU fit times were approximately 12.3 seconds for the H2-style linear cap, 33.1 for fixed nonlinear PC, 33.1 for H3 PC, 31.5 for generated BP and 15.3 for generated linear PC. They exclude shared base/cache work, dictionary construction, structural search and final evaluation; ancillary jobs sometimes overlapped. They support no end-to-end speed or hardware-efficiency claim.

12.3 What has not been established

- No independent-corpus generalization result is yet available for H3.
- No catastrophic-forgetting or continual-task-stream test is included in these H3 studies. Keeping reader tables does not freeze a neural routing function or guarantee retention.
- No decentralized training, frontier-scale throughput or large-model co-distillation is demonstrated.
- The observer grammar is bounded and hand-specified. Exact CD quotients, complete STLM search and a full RBII compression theorem are not implemented by these count tables.
- The final prediction stays inside a reader mixture. The cap does not write an arbitrary new byte distribution outside that convex hull.
- The main search is staged. Repeated neural/structural alternation remains a proposed experiment.

13 Why H3 is more successful than H1: evidence and interpretation

A careful answer has two levels.

Supported by matched H3 controls: generated readers improve the fixed-reader nonlinear cap; the child improves over its parent alone; selected programs beat the random-addition control; causal settling affects predictions; and a larger fitting set makes the nonlinear-versus-linear comparison more favorable. These are directly tested relationships within H3’s setup.

Plausible explanation of the cross-version change: H3’s readers supply explicit predictive evidence, its new programs fit meaningful conditional distributions, and the cap can learn context-dependent trust before reconciling that trust with recent observed outcomes. H1 asked a comparatively small residual network to turn hashed feature identities into corrections, while offering less additional evidence at inference. The new interface and memory therefore reduce a plausible representation and estimation burden.

Several changes are confounded across versions: dictionary size and training statistics, feature representation, cap design, objectives, search grammar, fitting schedules and inference. There is no one-factor H1-to-H3 experiment that proves a unique cause. In particular, H2 already achieved most of the large gain over H1 with a linear gate. H3’s further contribution is the more convincing demonstration of reader creation and composition, plus a nonlinear route that becomes more competitive with more data.

It would be premature to say that the nonlinear PC mechanism alone explains H3’s success. The same-reader BP model is close, and the same-reader linear PC model is strong. The more defensible statement is that H3 makes the desired mechanisms useful enough to warrant a larger, better-controlled test.

14 The supplied 6.5-hour Colab experiment

14.1 Question and scale

The new notebook tests the staged H3 recipe on enwik8, a 100-million-byte Wikipedia text/markup corpus with a published checksum [5]. It trains an independent 7.56-million-parameter byte transformer from scratch, freezes it, generates readers from training data and compares seven cap conditions across three seeds. It is intended to answer whether the sandbox mechanisms remain useful with a stronger base and substantially more fitting data, not whether a frontier model can already be replaced.

Setting	H3 sandbox primary / follow-up	New Colab default
Corpus	1.115 MB Tiny Shakespeare	100 MB enwik8
Base	120,576 parameters; 2 blocks; width 64	7,562,240 parameters; 6 blocks; width 320
Base context	64 bytes	256 bytes; score latter 128
Reader fitting prefix	700,000 bytes	4,000,000 bytes
Generated observer window	64 bytes	64 bytes, retained deliberately
Cap-fitting targets	16,384 / 65,536	262,144
Outcome targets	4,096	Pool 16,384; screen 8,192
Cap hidden widths	96, 64	128, 96
Maximum generated keys	12,000 per reader	32,768 per reader
Code-proxy ceiling	16 million bits per reader	32 million bits per reader
Maximum births	4	4; no birth is forced
Cap updates	480	Hardware-calibrated common count; at most 12,000
Cap seeds	5 / 3	3

Table 9: A scaled H3 experiment, not an exact numerical replication. Search and memory changes are explicit.

The seven conditions are fixed linear PC, fixed nonlinear PC, generated linear PC, H3 PC, same-architecture BP, first-generated-reader-only PC, and generated nonlinear PC retrained without prefix settling. The random-addition control is omitted to keep the main run within its budget. Baseline scores also include the frozen base, an EM reader mixture and a validation-selected single reader.

A later-birth gain is evidence about composition only when search actually chooses a child. If it chooses independent roots, the first-only comparison tests additional readers more generally. If no reader passes, or only one passes, some controls coincide; the notebook records this rather than manufacturing structural success.

14.2 Data roles and evaluation

The first 90 MB train the base. The first 4 MB also fit fixed and generated reader counts; gate-fitting blocks occur after that prefix. The next 5 MB are divided into base-selection and cap-selection halves. The final 5 MB are held out, with 262,144 targets sampled in nonoverlapping contexts spread across that region. Cap selection uses 16,384 targets. Search screens 16,384 fitting targets and 8,192 outcome targets; final cap fitting uses the full 262,144-target fitting set.

These are selected-block bpb scores, not full-stream enwik8 leaderboard scores. All arms are paired on the same positions. Test outcomes are evaluated after final model choices are locked, or after a clearly marked partial-model lock if the budget runs out. No test outcome changes the training recipe.

The base-training allowance is at most 2.25 hours. The remaining active-time budget covers dictionary and observer construction, proposal search, feature caching, cap training and evaluation. Once preprocessing is complete, measured update/evaluation times select one common cap-update budget, reserving 30 minutes for final work. Equal update counts do not mean equal floating-point work or elapsed time.

14.3 Persistence and progress

The notebook stores corpus bytes, code, splits, observer vocabulary and matrix, every candidate, accepted reader tables, cache shards, optimizer states, selected models, per-target losses and result summaries in the selected Google Drive folder. It retains earlier checkpoint versions and performs no automatic cleanup.

A payload is copied, hashed and verified before a separate immutable commit record is published. On resume, incomplete writes are ignored and a corrupt newest committed payload falls back to an older valid one. Base checkpoints occur approximately every five minutes, cap checkpoints approximately every two minutes, with saves at completed stages and each candidate/cache shard. Training reports progress about every 30 seconds.

The default limit is 6.5 recorded active hours across sessions, with at most seven allowed. Disconnected time is excluded. A hard reset can lose the short unlogged interval, and a network stall or individual save can outlast a timing check. This is a safe-boundary budget, not a guaranteed external wall-clock limit. Colab resources and runtime availability vary [6].

14.4 What was tested before delivery

The delivered implementation passes 18 numerical, causal, persistence and integration tests. They cover the inherited readers, nonlinear local gradients and weak-nudge behavior, empty-reader backoff, actual child proposal creation, source/configuration locks, exact PC/BP optimizer continuation, and target/future edits that leave current prediction inputs unchanged.

Every actual notebook code cell was executed in order in two fresh CPU processes using a local persistence directory and a reduced 500,000-byte Tiny Shakespeare input. The first run completed seven conditions at two seeds, discovered two readers (a lag-pair and its word-length refinement), and evaluated them. The resumed run preserved all 122 checkpoint payloads and returned an identical completed report. These tiny-run scores are software checks, not added H3 research evidence.

A separate injected training deadline exercised partial evaluation after four completed conditions, followed by completion with the unchanged recipe; final scores matched the uninterrupted run. Google Drive authentication/FUSE and the full multi-hour GPU run were not exercised in this sandbox.

14.5 How to run and return results

Upload `H3_enwik8_Colab_6p5h.ipynb` to Colab, select a GPU, keep the default run name or choose a fresh one, and run all cells. On interruption, reconnect Drive and rerun the same notebook with the same name and configuration. The default folder is

```
MyDrive/CDPC_Experiments/H3_enwik8_v1/.
```

The compact `results/return_results.zip` contains the report, search records, configuration, source, progress and per-target losses. Return that ZIP for analysis; full model/data checkpoints remain in the Drive folder. A partial status bundle is produced even if preprocessing stops early. Do not run concurrent notebook copies against the same run folder.

15 Assessment and next decisions

The most useful next observation is whether generated readers improve a strong fixed-reader nonlinear cap on a new corpus, while a generated-reader linear cap and same-architecture BP remain competitive controls. A positive result on these comparisons would support reader generation as a scalable module and clarify whether nonlinear activity inference adds value beyond the memory interface.

If scalar search accepts no useful candidates, the recorded trials can distinguish sparse support, size ceilings and weak insertion gains. The development late-birth result suggests contextual refitting as the next search objective to test; it does not justify silently replacing a failed screen with hand-selected Shakespeare readers.

If H3 beats the fixed bank but not generated linear PC, the reader-generation idea can still be useful while the nonlinear cap requires more evidence. If BP matches or beats PC, the reader architecture remains viable and PC-specific claims should narrow accordingly. If all learned caps merely track the global mixture, the new memory interface is doing most of the work.

H3 is therefore a meaningful step from operational components to useful structural contributions. The remaining questions concern how much nonlinear state inference earns, how search should evaluate conditional utility, how many independent search states a historical guide needs, and whether those benefits persist when the base, data and costs are scaled together.

A Evidence locations and reproducibility

The original sandbox release is `H3_dynamic_readers_sandbox.zip`. Its main evidence includes:

- `results/main/analysis.json`: five-seed primary means, seed gains and block intervals.
- `results/larger_fit/analysis.json`: the separate 65,536-target follow-up.
- `results/search.json`: accepted programs and all proposal screens.
- `results/late_birth.json` and `results/pc_residual_search.json`: the refit demonstration and failed scalar screen.
- `results/diagnostics.json`, `memory_accounting.json` and `evidence_verification.json`: numerical, storage and aggregation audits.
- Saved per-target arrays and model checkpoints under the primary and follow-up result directories.

The tables and plotted intervals in this paper are generated from the retained analysis JSON. The source bundle includes those compact evidence files, the figure-generation script, ASCII LaTeX and compile instructions. The separate Colab bundle includes the notebook, inspectable Python source and test logs. It does not duplicate the full 50 MB original sandbox release.

References

- [1] CD / PC research working files. *CD / predictive-coding hybrid: sandbox results* (H1), with implementation and per-target evidence. Prior project artifact, 2026.
- [2] CD / PC research working files. *PC-CD v2: sandbox results* (H2), with implementation and per-target evidence. Prior project artifact, 2026.
- [3] CD / PC research working files. *H3 sandbox: generated CD readers and nonlinear predictive coding*. Original release and verification records, September 2026.
- [4] B. Scellier and Y. Bengio. *Equilibrium Propagation: Bridging the Gap Between Energy-Based Models and Backpropagation*. [arXiv:1602.05179](https://arxiv.org/abs/1602.05179), 2016/2017.
- [5] M. Mahoney. *About the Test Data*. Corpus description and official enwik8 checksums. <https://mattmahoney.net/dc/textdata.html>. Accessed September 2026.
- [6] Google. *Colaboratory FAQ*. Runtime availability and Drive guidance. <https://research.google.com/colaboratory/faq.html>. Accessed September 2026.